Master: Bases de Données Fall, 2026

TP 1 (universityDB-data)

1 Setting up PostgreSQL

1. USING USPN LABS DATABASE

- Login to `linux` with your user account.
- Type the following command to start the `postgreSQL prompt`:

  ```
  psql -h postgres -U m1info_xx
  ```

  where `xx` is your account number between 1 and 90 $(1, 2, \ldots, 9, 10, \ldots, 90)$. The password is `M1INFO_xx`, with the same value of `xx` as your account number.
- You can see the list of tables present by typing :

  ```
  \d
  ```
- Download the following file in the same directory from which you ran the `psql` command:

  universityDB-schema.sql.

  This file contains the `SQL` code to create tables (I have given this code at the end of this subject).
- To read all the commands in the above file with `psql`, type :

  ```
  \i universityDB-schema.sql
  ```

  If this works, then on the left you can see the list of all the tables created.
- Once your tables are created, you can enter the data from the file universityDB-data.sql and run it. This file contains the `SQL` code to add data to our university database tables.

  ```
  \i universityDB-data.sql
  ```
- Now you can run whatever queries you want, for example :

  ```
  SELECT * FROM teacher;
  ```

Note: if you want to delete all the tables (*do this only if you did not create tables correctly*), you can run the following code.

```
drop table prereq;
drop table time_slot;
drop table advisor;
drop table takes;
drop table student;
drop table teaches;
drop table section;
drop table teacher;
drop table course;
drop table department;
drop table classroom;
```

2 Exercises

Now write the `SQL` code, and then run to verify, the following queries. The table after each question gives the result that you should get.

a.) Find the total enrollment of each course/section/semester/year.

Trouvez l'effectif total de chaque cours (par `section_id`, `semestre`, et `year`).

course_id	*sec_id*	*semester*	*year*	*count*
BIO-301	1	Summer	2010	1
CS-101	1	Fall	2009	6
CS-190	2	Spring	2009	2
CS-315	1	Spring	2010	2
BIO-101	1	Summer	2009	1
CS-101	1	Spring	2010	1
CS-319	2	Spring	2010	1
EE-181	1	Spring	2009	1
HIS-351	1	Spring	2010	1
FIN-201	1	Spring	2010	1
MU-199	1	Spring	2010	1
CS-347	1	Fall	2009	2
CS-319	1	Spring	2010	1
PHY-101	1	Fall	2009	1

```
SELECT takes.course_id, takes.sec_id, takes.semester, takes.year, count(*)
FROM takes
GROUP BY takes.course_id, takes.sec_id, takes.semester, takes.year;
```

b.) List all pairs of students who have taken at least one class together. Do **not** use **GROUP BY**.

Énumérez tous les couples d'étudiants qui ont suivi au moins un cours ensemble. Ne utilisez pas **GROUP BY**.

name	*name*
Bourikas	Brown
Bourikas	Levy
Bourikas	Shankar
Bourikas	Williams
Bourikas	Zhang
Brown	Levy
Brown	Shankar
Brown	Williams
Brown	Zhang
Levy	Shankar
Levy	Williams
Levy	Zhang
Shankar	Williams
Shankar	Zhang
Williams	Zhang

```
SELECT distinct S1.name, S2.name
FROM student as S1, takes as T1,
student as S2, takes as T2
WHERE S1.id = T1.id AND
S2.id = T2.id AND
T1.course_id = T2.course_id AND
T1.sec_id = T2.sec_id AND
T1.semester = T2.semester AND
T1.year = T2.year AND
S1.id < S2.id;
```

c.) Find courses taught in consecutive semesters.

Trouver des cours enseignés en semestres consécutifs.

title	*semester1*	*year1*	*semester2*	*year2*
Intro. to Computer Science	Fall	2009	Spring	2010

```
SELECT c.title, s1.semester as semester1, s1.year as year1
, s2.semester as semester2, s2.year as year2
FROM course AS c, section AS s1, section AS s2
WHERE c.course_id = s1.course_id
AND c.course_id = s2.course_id
AND ((s1.year = s2.year AND
((s1.semester = 'Fall' AND s2.semester = 'Spring') OR
(s1.semester = 'Spring' AND s2.semester = 'Fall'))) OR
(s1.year + 1 = s2.year AND s1.semester = 'Fall' AND s2.
semester = 'Spring'));
```

d.) Compute the sum of all credits over all courses for each year. The number of credits for each course is available in the table `course`.

Calculez la somme de tous les crédits sur tous les cours pour chaque année.
Le nombre de crédits pour chaque cours est disponible dans la table `course`.

year	*sum*
2010	29
2009	49

```
SELECT year, SUM(credits)
FROM takes, course
WHERE takes.course_id = course.course_id
GROUP BY year;
```

e.) For each department, find the total number of times a course was taught in that departments building by that departments teachers.

Pour chaque département, trouvez le nombre total de fois où un cours a été enseigné dans le bâtiment de ce département par les enseignants de ce département.

dept_name	*count*
Comp. Sci.	4
Elec. Eng.	1
History	1
Music	1
Physics	1

```
SELECT department.dept_name, count(*)
FROM section, department, teacher, teaches
WHERE (section.course_id, section.sec_id, section.semester
, section.year) = (teaches.course_id, teaches.sec_id,
teaches.semester, teaches.year) and
teaches.id = teacher.id and teacher.dept_name = department
.dept_name and department.building = section.building
GROUP BY department.dept_name;
```

f.) Output the name of each student and the sum of credits (from table `course`) taken in courses that are *not* from his department.

Affichez le nom de chaque étudiant et la somme des crédits (provenant de la table `course`) obtenus dans des cours qui ne sont *pas* de son département.

name	*sum*
Bourikas	7
Levy	11

```
SELECT student.name, sum(credits)
FROM student, course, takes
WHERE student.id = takes.id and takes.course_id = course.
course_id and student.dept_name != course.dept_name
GROUP BY student.name;
```

universityDB-schema.sql

```
create table classroom
  (building    varchar(15),
   room_number    varchar(7),
   capacity   numeric(4,0),
   primary key (building, room_number)
  );

create table department
  (dept_name    varchar(20),
   building   varchar(15),
   budget          numeric(12,2),
   primary key (dept_name)
  );

create table course
  (course_id    varchar(8),
   title      varchar(50),
   dept_name    varchar(20),
   credits    numeric(2,0),
   primary key (course_id),
   foreign key (dept_name) references department
  );

create table teacher
  (ID     varchar(5),
   name     varchar(20),
   dept_name    varchar(20),
   salary     numeric(8,2),
   primary key (ID),
   foreign key (dept_name) references department
  );

create table section
  (course_id    varchar(8),
       sec_id     varchar(8),
   semester   varchar(6)
    check (semester in ('Fall', 'Winter', 'Spring', 'Summer')),
   year     numeric(4,0),
   building   varchar(15),
   room_number    varchar(7),
   time_slot_id   varchar(4),
   primary key (course_id, sec_id, semester, year),
   foreign key (course_id) references course,
   foreign key (building, room_number) references classroom
  );

create table teaches
  (ID     varchar(5),
   course_id    varchar(8),
   sec_id     varchar(8),
   semester   varchar(6),
   year     numeric(4,0),
   primary key (ID, course_id, sec_id, semester, year),
   foreign key (course_id,sec_id, semester, year) references section,
   foreign key (ID) references teacher
  );

create table student
  (ID     varchar(5),
   name     varchar(20),
   dept_name    varchar(20),
   tot_cred   numeric(3,0),
   primary key (ID),
   foreign key (dept_name) references department
  );

create table takes
  (ID     varchar(5),
   course_id    varchar(8),
   sec_id     varchar(8),
   semester   varchar(6),
   year     numeric(4,0),
   grade          varchar(2),
   primary key (ID, course_id, sec_id, semester, year),
   foreign key (course_id,sec_id, semester, year) references section,
   foreign key (ID) references student
  );

create table advisor
  (s_ID     varchar(5),
   i_ID     varchar(5),
   primary key (s_ID),
   foreign key (i_ID) references teacher (ID),
   foreign key (s_ID) references student (ID)
  );

create table time_slot
  (time_slot_id   varchar(4),
   day      varchar(1),
   start_hr   numeric(2),
   start_min    numeric(2),
   end_hr     numeric(2),
   end_min    numeric(2),
   primary key (time_slot_id, day, start_hr, start_min)
  );

create table prereq
  (course_id    varchar(8),
   prereq_id    varchar(8),
   primary key (course_id, prereq_id),
   foreign key (course_id) references course,
   foreign key (prereq_id) references course
  );
```

Tables

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

(a) course

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

(b) teacher

id	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

(c) student

course_id	*sec_id*	*semester*	*year*	*building*	*rn*	*time_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

(d) section

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

(e) teaches

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

(f) takes

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

(g) department

building	*room_number*	*capacity*
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

(h) classroom

3 ONLY IF LINUX DOES NOT WORK

Here is how to use `PostgreSQL` online.

2. USING ONLINE DATABASE

- Go to the site

  `https://sqliteonline.com/`

- On the left, choose '`PostgreSQL`' and click on '`Click to connect`'.

- In the main window, create the database for the university by copy-pasting and running the code in file universityDB-schema.sql. This file contains the `SQL` code to create tables (I have given this code at the end of this subject).

  If this works, then on the left you can see the list of all the tables created.

- Once your tables are created, you can enter the data from the file universityDB-data.sql and run it. This file contains the `SQL` code to add data to our university database tables (I have given this code at the end of this subject).